

To-Do List Application - API Documentation

1. Introduction

This document describes a simple RESTful API for a To-Do List application. The application allows users to create tasks, view all tasks, view a single task, update task information, and delete tasks.

The focus of this task is API design and documentation, not backend implementation.

2. Objectives

- allow users to add new tasks
- retrieve all tasks
- retrieve a single task by ID
- update selected fields of a task
- delete a task
- handle errors clearly and consistently
- follow RESTful API design principles

3. REST API Overview

REST is a style of designing APIs where resources are represented using URLs and standard HTTP methods are used to perform operations on those resources.

In this project, a task is the main resource.

4. Task Data Model

```
{
  "id": 1,
  "title": "Complete assignment",
  "description": "Finish the API documentation",
  "completed": false,
  "createdAt": "2026-10-06T10:00:00Z",
```

```
"updatedAt": "2026-10-06T10:00:00Z"
}
```

Field	Type	Description
id	integer/string	A unique identifier for the task. It should be generated by the backend or database, not manually by the client.
title	string	The short name or heading of the task.
description	string	Extra detail about the task.
completed	boolean	Indicates whether the task is completed.
createdAt	string	Date and time when the task was created.
updatedAt	string	Date and time when the task was last updated.

Important note: the client should not manually generate the task ID. The backend or database should generate it automatically.

5. API Endpoint Summary

Operation	HTTP Method	Endpoint	Purpose
Create task	POST	/api/tasks	Add a new task
Get all tasks	GET	/api/tasks	Retrieve all tasks
Get one task	GET	/api/tasks/:id	Retrieve one task by ID
Update task	PATCH	/api/tasks/:id	Modify one or more task fields
Delete task	DELETE	/api/tasks/:id	Remove a task

6. Create Task API

Endpoint: POST /api/tasks

Purpose: Create a new task.

Request body:

```
{
  "title": "Complete assignment",
  "description": "Finish the API documentation",
  "completed": false
}
```

Example request:

```
POST /api/tasks
Content-Type: application/json

{
  "title": "Complete assignment",
  "description": "Finish the API documentation",
  "completed": false
}
```

Example success response:

```
{
  "id": 1,
  "title": "Complete assignment",
  "description": "Finish the API documentation",
  "completed": false,
  "createdAt": "2026-10-06T10:00:00Z",
  "updatedAt": "2026-10-06T10:00:00Z"
}
```

Success status: 201 Created

Error response example:

```
{
  "error": "Title is required"
}
```

Error status: 400 Bad Request

Reasoning: POST is used because we are creating a new resource.

7. Get All Tasks API

Endpoint: GET /api/tasks

Purpose: Retrieve all tasks.

Example request:

```
GET /api/tasks
```

Example success response:

```
[
  {
    "id": 1,
    "title": "Complete assignment",
    "description": "Finish the API documentation",
    "completed": false,
    "createdAt": "2026-10-06T10:00:00Z",
    "updatedAt": "2026-10-06T10:00:00Z"
  }
]
```

Success status: 200 OK

Error status: 500 Internal Server Error

Reasoning: GET is the correct HTTP method for retrieving data.

8. Get Single Task API

Endpoint: GET /api/tasks/:id

Example: GET /api/tasks/1

Purpose: Retrieve one specific task using its unique ID.

Path parameter: :id = the unique ID of the task.

Example success response:

```
{
  "id": 1,
  "title": "Complete assignment",
  "description": "Finish the API documentation",
  "completed": false,
  "createdAt": "2026-10-06T10:00:00Z",
  "updatedAt": "2026-10-06T10:00:00Z"
}
```

Success status: 200 OK

Error response example:

```
{
  "error": "Task not found"
}
```

Error status: 404 Not Found

Reasoning: GET is used because we are reading an existing resource.

9. Update Task API

Endpoint: PATCH /api/tasks/:id

Purpose: Update one or more fields of an existing task.

Request body:

```
{
  "title": "Complete API assignment",
  "completed": true
}
```

Example request:

```
PATCH /api/tasks/1
Content-Type: application/json

{
```

```
"title": "Complete API assignment",
"completed": true
}
```

Example success response:

```
{
  "id": 1,
  "title": "Complete API assignment",
  "description": "Finish the API documentation",
  "completed": true,
  "createdAt": "2026-10-06T10:00:00Z",
  "updatedAt": "2026-10-06T10:18:00Z"
}
```

Success status: 200 OK

Error status: 404 Not Found

Reasoning: PATCH is used when we want to modify only some fields of an existing task.

10. Delete Task API

Endpoint: DELETE /api/tasks/:id

Purpose: Delete an existing task.

Example request:

```
DELETE /api/tasks/1
```

Success status: 204 No Content

Error response example:

```
{
  "error": "Task not found"
}
```

Error status: 404 Not Found

Reasoning: DELETE is appropriate because the purpose is to remove a resource from the system.

11. CRUD Operations

- **CREATE:** POST /api/tasks
- **READ:** GET /api/tasks and GET /api/tasks/:id
- **UPDATE:** PATCH /api/tasks/:id
- **DELETE:** DELETE /api/tasks/:id

12. HTTP Status Codes

Status Code	Meaning	When to Use It
200 OK	Request succeeded	Used for successful GET and PATCH responses
201 Created	A resource was created	Used after successful POST
204 No Content	Request succeeded with no body returned	Used after successful DELETE
400 Bad Request	Client sent invalid data	Used when required fields are missing or values are wrong
404 Not Found	Resource was not found	Used when the task ID does not exist
500 Internal Server Error	Server error	Used when something unexpected happens on the server

13. Validation Rules

- title is required
- title cannot be empty
- completed must be a boolean value
- id must uniquely identify a task
- server or database generates the ID

Example error:

```
{  
  "error": "Title is required"  
}
```

14. Error Handling

400 Bad Request: invalid request data

404 Not Found: task ID is missing or invalid

500 Internal Server Error: unexpected server error

15. Unique ID Generation

Each task must have a unique ID so it can be edited or deleted reliably.

The client should not assign IDs manually because duplicate IDs can cause conflicts.

The backend or database should generate a unique ID automatically.

Example UUID:

```
"id": "a7c6b1d0-18e3-4d08-b8e2-aa1234bb5678"
```

16. REST Design Reasoning

This design follows REST principles because tasks are treated as resources, endpoints are meaningful, HTTP methods are used correctly, and JSON is used for request and response bodies.

The `:id` path parameter represents the unique ID of the task being accessed.

Consistent endpoint naming makes the API easier to understand.

17. Potential Implementation Challenges

- Input validation
- Unique task IDs
- Task not found
- Database/data persistence

- Concurrent updates
- Error handling
- Authentication/authorization for multiple users
- Frontend-backend integration
- Invalid request data
- Server/database failures

18. Conclusion

This To-Do List API design follows RESTful principles and uses the standard CRUD flow. It is beginner-friendly, consistent, and suitable for a basic task management application.